

Exercice 3 — Pool de threads pour le traitement de requêtes

Thème : *ExecutorService, threads réutilisables, file d'attente sous charge*

Contexte

On souhaite illustrer, en Java pur (sans framework), comment un pool de threads permet de traiter plusieurs tâches (simulant des requêtes HTTP) sans créer un thread par tâche.

Objectifs

- Créer et utiliser un `ExecutorService` à taille fixe.
- Soumettre plusieurs tâches concurrentes et observer la réutilisation des threads.
- Fermer correctement le pool de threads.

Travail demandé

- 1 Créez un `ExecutorService` avec un pool fixe de 4 threads via `Executors.newFixedThreadPool(4)`.
- 2 Soumettez 10 tâches (`Runnable`) qui simulent le traitement d'une requête : afficher le nom du thread courant, puis attendre 500 ms (`Thread.sleep()`).
- 3 Observez et expliquez pourquoi seulement 4 noms de threads distincts apparaissent dans la sortie, même si 10 tâches ont été soumises.
- 4 Fermez proprement l'`ExecutorService` (`shutdown()` puis attente de la terminaison), pour éviter que l'application ne reste bloquée.
- 5 Expliquez, en une ou deux phrases, pourquoi un serveur web utilise un mécanisme équivalent pour traiter les requêtes HTTP entrantes plutôt que de créer un thread par requête.

Critères d'évaluation

- Le pool est correctement dimensionné et utilisé pour soumettre les tâches.
- L'`ExecutorService` est fermé proprement (pas de fuite de threads).
- L'explication sur la réutilisation des threads est correcte.